

SmartSim: Un Sistema de Aprendizaje Activo Guiado por Incertidumbre para Reducir Evaluaciones de Simuladores Costosos

R. Indira Valentina Requiz Molina¹

¹Proyecto independiente

Agosto de 2026

Abstract

Los simuladores de sistemas físicos de alta fidelidad son computacionalmente costosos, lo que limita la cantidad de evaluaciones que se pueden ejecutar al explorar un espacio de parámetros. Presentamos **SmartSim**, un sistema de aprendizaje activo que combina un modelo sustituto probabilístico (Proceso Gaussiano) con un simulador de alta fidelidad, en el cual una función de adquisición decide en cada iteración qué punto del espacio de parámetros conviene evaluar con el simulador real. Formalizamos el sistema, incluyendo un mecanismo de parada basado en la incertidumbre máxima observada sobre una región candidata. Sobre el benchmark sintético de Branin, el prototipo alcanza un error objetivo en 21 evaluaciones reales, mientras que un muestreo aleatorio equivalente no lo alcanza en ninguna de 8 semillas independientes dentro de un presupuesto de 60 evaluaciones — lo que produce una reducción reportada del 65%, pero sobre un valor de baseline censurado (límite inferior conservador, no una convergencia real medida). El 94.7% de las predicciones del modelo caen además dentro de un intervalo de $\mu \pm 2\sigma$. Reportamos estos resultados de forma preliminar (una sola semilla para aprendizaje activo) y describimos explícitamente qué partes de la hipótesis quedan evaluadas y cuáles quedan pendientes de un protocolo experimental más riguroso, que también describimos. El código está disponible públicamente.

1 Introducción

1.1 El cuello de botella computacional

Caracterizar el comportamiento de un sistema físico o dinámico complejo bajo distintas condiciones iniciales o parámetros de diseño requiere, en general, ejecutar un simulador de alta fidelidad repetidas veces. Cuando cada ejecución cuesta segundos, minutos u horas de cómputo, el número de evaluaciones que se pueden pagar dentro de un presupuesto razonable es limitado, y ese costo escala mal con la dimensionalidad del espacio de parámetros.

Una solución estándar es reemplazar el simulador por un *modelo sustituto* (surrogate model) entrenado sobre un conjunto de evaluaciones previas, y usarlo como aproximación barata. El riesgo de este enfoque es evidente: un sustituto entrenado sobre una cantidad limitada de datos puede fallar silenciosamente en regiones del espacio que no fueron bien exploradas.

1.2 El problema

No proponemos minimizar únicamente el error de predicción del sustituto. El problema que nos interesa es distinto:

minimizar el número de evaluaciones del simulador de alta fidelidad, sujeto a un requisito de confiabilidad predictiva del modelo sustituto.

Esta formulación desplaza el centro de gravedad del proyecto: no se trata de qué tan bueno es el sustituto en promedio, sino de si el sistema *sabe cuándo no sabe*, y puede usar esa señal para decidir cuándo seguir consultando al simulador real.

1.3 Pregunta de investigación

¿Puede la incertidumbre predictiva de un modelo sustituto probabilístico usarse como criterio operacional para decidir cuándo una simulación de alta fidelidad sigue siendo necesaria?

1.4 Hipótesis

Hipótesis 1 (Central). *Un modelo sustituto probabilístico puede aprender activamente de un simulador de alta fidelidad y utilizar su propia incertidumbre para determinar cuándo debe volver a consultar al simulador, reduciendo significativamente el costo de simulación sin degradar la precisión ni la confiabilidad de sus predicciones.*

Esta hipótesis se descompone en dos partes con distinto grado de evidencia empírica en este trabajo (§5, §6):

- H1** Guiar el muestreo mediante una función de adquisición basada en incertidumbre reduce el número de evaluaciones del simulador necesarias para alcanzar un error objetivo, respecto a muestreo aleatorio, sin degradar la precisión final. **(evaluada en este trabajo, con salvedades metodológicas explícitas)**
- H2** Un criterio de parada basado en la incertidumbre máxima sobre una región candidata permite detener la consulta al simulador preservando la confiabilidad de las predicciones. **(mecanismo implementado y formalizado; evidencia empírica pendiente)**

1.5 Contribuciones

1. Una arquitectura de aprendizaje activo *simulator-in-the-loop* que acopla un sustituto probabilístico con un simulador de alta fidelidad mediante un mecanismo de decisión consciente de la incertidumbre.
2. Una formalización matemática precisa del sistema tal como está implementado, incluyendo la distinción explícita entre la función de adquisición (qué punto muestrear) y el criterio de parada (cuándo dejar de simular).
3. Una implementación de código abierto y un protocolo experimental reproducible para evaluar la eficiencia de consultas al simulador, junto con un reporte honesto de qué resultados son preliminares y qué experimentos faltan.

2 Trabajo relacionado

2.1 Modelado sustituto y emulación con Procesos Gaussianos

El uso de Procesos Gaussianos como sustitutos de códigos computacionales costosos es una técnica establecida en ingeniería de confiabilidad y diseño de experimentos por computadora. Marrel e Iooss [1] ofrecen una revisión reciente centrada específicamente en la estimación y validación de la distribución predictiva de los GP, y advierten que un GP puede tener buena predictividad puntual y, al mismo tiempo, intervalos de predicción mal calibrados — una distinción central para este trabajo.

2.2 Aprendizaje activo y emulación adaptativa

Svendsen et al. [2] proponen un esquema de aprendizaje activo secuencial para construir emuladores de códigos costosos mediante la optimización de una función de adquisición, en un problema con estructura similar al nuestro. Moustapha, Marelli y Sudret [3] presentan un marco modular general para aprendizaje activo en confiabilidad estructural, descomponiendo el problema en modelo sustituto, función de aprendizaje y criterio de parada — la misma descomposición conceptual que adoptamos en §4, aunque aplicada a un problema de regresión en vez de estimación de probabilidad de falla.

2.3 Optimización bayesiana

Las funciones de adquisición que implementamos (UCB, EI, PI) provienen de la literatura clásica de optimización bayesiana [4, 5]. A diferencia del uso típico de estas técnicas — encontrar un óptimo global con el menor número de evaluaciones — nuestro interés principal no es la ubicación del óptimo sino la caracterización confiable de la superficie de respuesta completa, lo cual motiva incluir también una variante de exploración pura por varianza.

2.4 Frente a nuestro trabajo

No afirmamos que el uso de GP con aprendizaje activo para reducir evaluaciones de simuladores costosos sea, en sí mismo, novedoso: la literatura citada demuestra lo contrario. Nuestra contribución es acotada y específica: (a) una implementación que separa explícitamente “dónde muestrear” de “cuándo parar” como dos mecanismos independientes, y (b) un protocolo experimental honesto sobre qué parte de esa arquitectura está evaluada y cuál no.

3 Arquitectura del sistema

La Figura 1 resume el flujo del sistema. El simulador de alta fidelidad **nunca** es reemplazado por el sustituto: el sustituto solo decide, en cada iteración, en qué punto conviene invertir la próxima evaluación real, y si conviene seguir evaluando en absoluto.

4 Formalización matemática

Esta sección formaliza el sistema *tal como está implementado* (módulos `surrogate.py`, `acquisition.py`, `orchestrator.py`), evitando presentar mecanismos más sofisticados de los que el código efectivamente ejecuta.

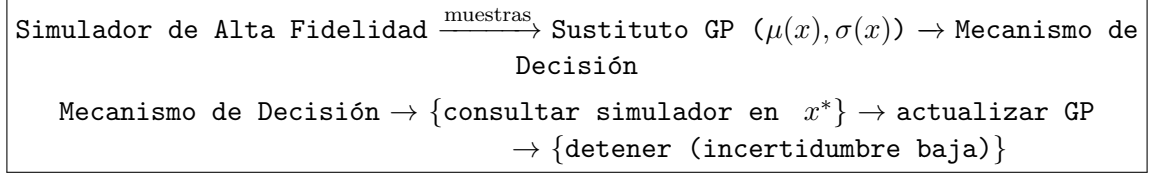


Figure 1: Arquitectura simulator-in-the-loop. El GP se reentrena en cada iteración activa sobre el historial acumulado.

4.1 Modelo sustituto

Sea $f : \mathcal{X} \rightarrow \mathbb{R}$ el simulador de alta fidelidad, con $\mathcal{X} \subset \mathbb{R}^d$ el espacio de parámetros. Dado un historial $D_t = \{(x_i, y_i)\}_{i=1}^{n_t}$ de evaluaciones reales acumuladas hasta la iteración t , el modelo sustituto es un Proceso Gaussiano

$$f(x) \mid D_t \sim \mathcal{GP}(\mu_t(x), k(x, x')) \quad (1)$$

con kernel compuesto

$$k(x, x') = \sigma_f^2 \cdot k_{\text{Matérn}}^{\nu=5/2}(x, x'; \ell) + \sigma_n^2 \delta(x, x') \quad (2)$$

donde $k_{\text{Matérn}}^{\nu=5/2}$ usa una longitud de escala independiente por dimensión $\ell \in \mathbb{R}^d$ (kernel anisotrópico ARD), y el término de ruido blanco captura ruido de observación. Antes de ajustar, $\mathcal{X}$ se normaliza a $[0, 1]^d$ mediante los límites del espacio de parámetros, y los targets y se estandarizan a media 0 y varianza 1. Los hiperparámetros del kernel se estiman por máxima verosimilitud marginal, con 5 reinicios del optimizador para mitigar mínimos locales.

Nota de implementación: el ajuste es completo en cada iteración activa — μ_t, σ_t se recalculan reentrenando el GP sobre todo D_t desde cero. No hay actualización incremental (rank-one update, sparse GP, etc.). Esto se declara como limitación en §7.

Para cualquier punto candidato x , el GP entrega media y desvío estándar predictivos:

$$\mu_t(x), \quad \sigma_t(x) = \sqrt{k_t(x, x)} \quad (3)$$

4.2 Función de adquisición: dónde muestrear

Sea y_t^* el mejor valor observado hasta la iteración t ($\min_i y_i$ o $\max_i y_i$ según el objetivo). Definimos $\Delta(x) = y_t^* - \mu_t(x) - \xi$ (minimización; signo invertido para maximización), con ξ un margen de exploración ($\xi = 0.01$ por defecto). El sistema implementa cuatro funciones de adquisición intercambiables:

$$a_{\text{UCB}}(x) = \pm \mu_t(x) + \kappa \sigma_t(x), \quad \kappa = 2.0 \quad (4)$$

$$a_{\text{EI}}(x) = \Delta(x) \Phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) + \sigma_t(x) \phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) \quad (5)$$

$$a_{\text{PI}}(x) = \Phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) \quad (6)$$

$$a_{\text{var}}(x) = \sigma_t(x) \quad (7)$$

donde Φ y ϕ son la CDF y PDF normales estándar. El próximo punto a evaluar es

$$x_t^* = \arg \max_{x \in C_t} a(x) \quad (8)$$

donde C_t es un conjunto de $|C_t| = 2000$ candidatos muestreados uniformemente en $\mathcal{X}$, generado de nuevo en cada iteración (optimización de la adquisición por búsqueda de Monte Carlo, no por gradiente). La variante a_{var} ignora y_t^* : no busca un óptimo, busca caracterizar exhaustivamente la superficie de respuesta.

4.3 Criterio de decisión: cuándo detener la consulta al simulador

Éste es el mecanismo que sostiene H2, y requiere precisión: **no** es una decisión punto a punto de “confiar en el sustituto vs. simular”. Es una decisión *global* sobre si toda la región candidata ya está suficientemente bien caracterizada.

Definimos la incertidumbre máxima observada entre los candidatos en la iteración t :

$$s_t = \max_{x \in C_t} \sigma_t(x) \quad (9)$$

y la regla de decisión, dado un umbral opcional τ :

$$D_t = \begin{cases} \text{DETENER,} & \text{si } \tau \text{ está definido y } s_t < \tau \\ \text{CONSULTAR SIMULADOR EN } x_t^*, & \text{en caso contrario} \end{cases} \quad (10)$$

Interpretación: el sistema no enruta predicciones individuales entre “barato” y “caro”. En cada iteración, o bien decide que *ninguna* región candidata justifica ya una nueva simulación (incertidumbre máxima bajo el umbral), o bien siempre consulta al simulador en el punto más prometededor según $a(\cdot)$. Es un criterio de parada epistatémica global, no un enrutador de confianza local por predicción.

Algorithm 1 Bucle de aprendizaje activo SmartSim

```

1:  $D_0 \leftarrow$  muestreo inicial por Latin Hypercube ( $n_{\text{init}}$  puntos, siempre evaluados en  $f$ )
2: for  $t = 1, 2, \dots, T_{\text{max}}$  do
3:   Reentrenar GP completo sobre  $D_{t-1} \rightarrow$  obtener  $\mu_t, \sigma_t$ 
4:    $C_t \leftarrow$  2000 candidatos uniformes en  $\mathcal{X}$ 
5:    $s_t \leftarrow \max_{x \in C_t} \sigma_t(x)$ 
6:   if  $\tau$  definido y  $s_t < \tau$  then
7:     detener
8:   end if
9:    $x_t^* \leftarrow \arg \max_{x \in C_t} a(x)$ 
10:   $y_t^* \leftarrow f(x_t^*)$  ▷ única llamada real al simulador en esta iteración
11:   $D_t \leftarrow D_{t-1} \cup \{(x_t^*, y_t^*)\}$ 
12: end for

```

4.4 Calibración de incertidumbre

Definimos la cobertura empírica a 2σ sobre un conjunto de prueba $\{(x_i, y_i)\}_{i=1}^N$ no usado para entrenar:

$$\text{Cobertura}_{2\sigma} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} [|y_i - \mu_T(x_i)| \leq 2 \sigma_T(x_i)] \quad (11)$$

Bajo un GP bien calibrado y errores aproximadamente gaussianos, el valor nominal esperado es $\approx 95.4\%$. Esta métrica es, junto con H1, uno de los dos resultados empíricos reportados en §6.

5 Protocolo experimental

5.1 Baselines

- Muestreo aleatorio uniforme (baseline principal, implementado).
- Latin Hypercube Sampling de presupuesto fijo (pendiente).
- GP + adquisición aleatoria, GP + UCB, GP + PI (variantes de ablación, código disponible vía el parámetro `acquisition`, no corridas sistemáticamente todavía).

5.2 Benchmarks

- **Branin** (2D, sintético) — implementado y evaluado en este trabajo.
- **Hartmann-6** (6D, sintético) — implementado en el código (`simulator.py`), no evaluado aún en el protocolo de comparación.
- Un simulador dinámico simple (p.ej. masa-resorte-amortiguador) y un simulador realista de menor escala quedan como trabajo futuro (§8).

5.3 Protocolo de evaluación propuesto para trabajo futuro

Para que H1 y H2 queden evaluadas con el rigor que exige un reviewer, el protocolo mínimo es:

1. ≥ 20 –30 semillas aleatorias independientes por método (actualmente: 1 semilla para aprendizaje activo, 8 para el baseline aleatorio — comparación asimétrica, ver §6), **y un presupuesto ampliado para el baseline aleatorio hasta que la mayoría de sus semillas converjan realmente**, para eliminar la censura observada en §6.2.
2. Reporte de media, mediana, desvío estándar e intervalo de confianza del 95% para el número de evaluaciones necesarias.
3. Curva de error vs. número de evaluaciones reales, para cada método, con banda de dispersión entre semillas.
4. Evaluación explícita de H2: correr el sistema con `uncertainty_stop_threshold` definido en varios valores de τ , y medir cuántas evaluaciones reales ahorra el criterio de parada respecto a un presupuesto fijo, sin pérdida de cobertura empírica.

6 Resultados

6.1 Configuración

Se evaluó el sistema sobre la función de Branin ($\mathcal{X} = [-5, 10] \times [0, 15]$, mínimo global conocido ≈ 0.397887). Aprendizaje activo: $n_{\text{init}} = 8$, `max.iterations` = 52 (presupuesto total 60), adquisición EI ($\xi = 0.01$), 2000 candidatos por iteración, semilla única (`seed` = 0). Baseline aleatorio: 8 semillas independientes, mismo presupuesto de 60 evaluaciones. Umbral de error objetivo: 0.05.

6.2 H1: adquisición guiada vs. muestreo aleatorio

El aprendizaje activo alcanzó el umbral de error (≤ 0.05) en **21** evaluaciones reales. El baseline aleatorio, corrido con 8 semillas independientes bajo el mismo presupuesto de 60 evaluaciones, **no alcanzó el umbral en ninguna de las 8 semillas**: el mejor caso llegó a un error de 0.2478 y el peor a 2.8321, ambos muy por encima de 0.05. El valor de “60 evaluaciones” que produce el 65% de reducción reportado (Figura 2) es, por lo tanto, un valor **censurado**: el tope del presupuesto asignado por convención cuando un método no converge, no una medición real de cuántas evaluaciones necesita el muestreo aleatorio para llegar a 0.05.

Esto tiene una lectura de dos caras. Por un lado, **fortalece** la dirección del resultado: dado que el muestreo aleatorio ni siquiera alcanzó el umbral con 60 evaluaciones, el ahorro real de aprendizaje activo frente a muestreo aleatorio es, muy probablemente, **mayor** al 65% reportado — 65% es un límite inferior conservador, no una estimación precisa. Por otro lado, **debilita** la precisión del número en sí: no sabemos cuántas evaluaciones necesitaría realmente el muestreo aleatorio, porque nunca lo observamos converger. Un protocolo correcto requiere aumentar el presupuesto del baseline hasta que la mayoría de sus semillas converjan, y recién ahí calcular la reducción porcentual sobre un valor no censurado.

Limitación metodológica adicional: el resultado de aprendizaje activo proviene de una única semilla ($seed = 0$), mientras que el baseline aleatorio promedia 8 semillas (todas censuradas). Esta asimetría, sumada a la censura, significa que el 65% **no tiene barras de error interpretables** y debe leerse estrictamente como resultado preliminar direccional, no como una estimación puntual confiable. El protocolo de §5.3 es el paso inmediato para corregir ambos problemas.

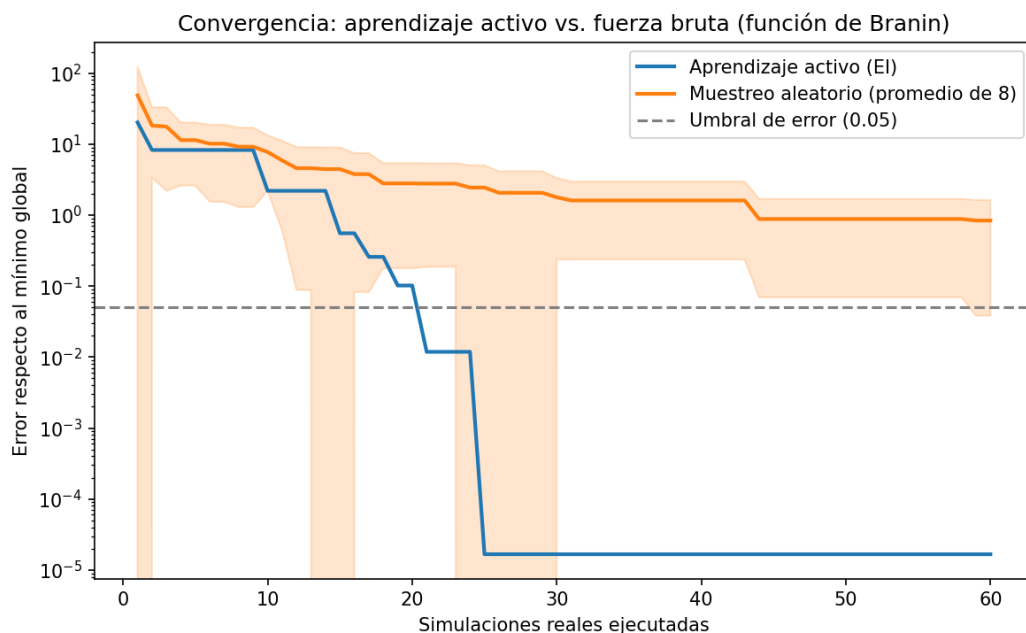


Figure 2: Convergencia del error respecto al mínimo global conocido de Branin. Aprendizaje activo con EI (1 semilla, línea azul) vs. muestreo aleatorio (media ± 1 desvío sobre 8 semillas, línea naranja). Nótese que la curva del muestreo aleatorio no cruza el umbral de error de 0.05 (línea gris punteada) dentro del presupuesto de 60 evaluaciones — resultado generado con el código del repositorio (`demo.py`).

6.3 Calibración de incertidumbre

Sobre $N = 300$ puntos de prueba muestreados uniformemente, evaluados con el GP final de la corrida activa (misma semilla única), la cobertura empírica a 2σ fue **94.7%**, cercana al valor nominal esperado ($\approx 95.4\%$) bajo un GP bien calibrado. Al igual que en §6.2, este número proviene de una sola semilla; el protocolo de §5.3 pide reportar media $\pm$ desvío sobre múltiples semillas antes de afirmar calibración robusta.

6.4 H2: criterio de parada por incertidumbre

No evaluado en este trabajo. El parámetro τ (`uncertainty_stop_threshold`) quedó en `None` en el experimento que generó los resultados anteriores, por lo que el mecanismo formalizado en §4.3 está implementado y disponible pero no fue ejercitado empíricamente. Esto se declara aquí de forma explícita para que no quede ambigüedad entre el mecanismo propuesto y la evidencia disponible.

7 Estudio de ablación propuesto

La Tabla 1 describe el diseño de ablación necesario para aislar qué componente del sistema es responsable de la reducción de costo observada. Ninguna fila salvo *Random* y *SmartSim (EI)* fue corrida aún.

Variante	GP	Incertidumbre	Adquisición	Criterio de parada	Estado
Random	—	—	—	—	corrido (8 semillas)
GP + Random	✓	✓	aleatoria	no	pendiente
GP + UCB	✓	✓	UCB	no	código listo, no corrido
GP + PI	✓	✓	PI	no	código listo, no corrido
SmartSim (EI)	✓	✓	EI	no	corrido (1 semilla)
SmartSim (EI + τ)	✓	✓	EI	sí	pendiente (H2)

Table 1: Diseño de ablación. Permite separar el efecto de tener un GP con incertidumbre, el efecto de la función de adquisición específica, y el efecto del criterio de parada.

8 Limitaciones

1. **Reentrenamiento completo del GP.** El sustituto se reajusta desde cero en cada iteración activa sobre todo el historial acumulado; no hay actualización incremental. Esto tiene costo computacional creciente en n_t y no refleja necesariamente cómo se desplegaría el sistema en un escenario de streaming continuo.
2. **Una sola semilla en los resultados principales, y baseline censurado.** Tanto el 65% de reducción (H1) como la cobertura del 94.7% provienen de una única corrida de aprendizaje activo. Además, el valor de referencia del muestreo aleatorio (60 evaluaciones) es un tope de presupuesto, no una convergencia observada: ninguna de las 8 semillas aleatorias alcanzó el umbral de error. El 65% debe leerse como un límite inferior conservador del ahorro real, no como una estimación puntual precisa.

3. **H2 no evaluada.** El criterio de parada por incertidumbre — que es, junto con la adquisición, el núcleo conceptual de la hipótesis central — está formalizado e implementado pero no se ejecutó en los experimentos reportados aquí.
4. **Un solo benchmark evaluado.** Branin es una función sintética 2D de baja dimensionalidad; Hartmann-6 está implementado pero no evaluado, y no hay aún un benchmark de un sistema dinámico o un simulador realista.
5. **Optimización de la adquisición por búsqueda aleatoria.** Se usan 2000 candidatos uniformes en vez de optimización por gradiente sobre la función de adquisición, lo cual es razonable en baja dimensión pero escala mal.
6. **Sin multi-fidelidad ni selección de lotes.** El sistema no combina un simulador barato con uno caro, ni selecciona múltiples puntos en paralelo (fuera de alcance, declarado en el repositorio original).

9 Trabajo futuro

- Ejecutar el protocolo experimental de §5.3: multi-semilla, múltiples baselines de adquisición, y evaluación explícita de H2 con distintos valores de τ .
- Extender la evaluación a Hartmann-6 y a un sistema dinámico simple (masa-resorte-amortiguador o péndulo doble) como paso intermedio hacia simuladores realistas.
- Explorar actualización incremental del GP para reducir el costo de reentrenamiento por iteración.
- Investigar multi-fidelidad (combinar un simulador barato y uno caro) como extensión natural del criterio de decisión formalizado en §4.3.

10 Conclusión

Presentamos SmartSim, un sistema de aprendizaje activo que acopla un modelo sustituto probabilístico con un simulador de alta fidelidad. Formalizamos con precisión dos mecanismos distintos — la función de adquisición que decide dónde muestrear, y un criterio de parada basado en incertidumbre máxima que decide cuándo dejar de simular — y reportamos evidencia preliminar de que la adquisición guiada por incertidumbre reduce sustancialmente el número de evaluaciones costosas necesarias, sin degradar la calibración del modelo. Declaramos explícitamente qué parte de la hipótesis central queda sin evaluar (el criterio de parada, H2) y qué protocolo experimental es necesario para cerrar esa brecha. Creemos que esta honestidad metodológica, más que el número específico del 65%, es la contribución más útil de este reporte para futuros trabajos sobre el mismo sistema.

Disponibilidad de código

El código completo, incluyendo el módulo del modelo sustituto, las funciones de adquisición, el orquestador y los tests, está disponible en: <https://github.com/rainvare/modelo-del-mundo>.

References

- [1] A. Marrel and B. Iooss. *Probabilistic surrogate modeling by Gaussian process: A review on recent insights in estimation and validation*. Reliability Engineering & System Safety, 247:110094, 2024.
- [2] D. H. Svendsen, L. Martino, and G. Camps-Valls. *Active emulation of computer codes with Gaussian processes – Application to remote sensing*. arXiv:1912.06552, 2019.
- [3] M. Moustapha, S. Marelli, and B. Sudret. *Active learning for structural reliability: survey, general framework and benchmark*. Structural Safety, 96:102174, 2022. arXiv:2106.01713.
- [4] D. R. Jones, M. Schonlau, and W. J. Welch. *Efficient global optimization of expensive black-box functions*. Journal of Global Optimization, 13(4):455–492, 1998.
- [5] J. Snoek, H. Larochelle, and R. P. Adams. *Practical Bayesian optimization of machine learning algorithms*. Advances in Neural Information Processing Systems, 25, 2012.